

CSC240_LinkedQueue_Practice/QueueType.h

```
1 // Header file for Queue ADT.
2 // Template version of the Linked Queue using a front and rear pointer.
3 #ifndef QUEUE_H
4 #define QUEUE_H
5 #include <new>
6 #include <iostream>
7 #include <cstdint>
8
9 template <class ItemType>
10
11 struct NodeType
12 {
13     ItemType info;
14     NodeType<ItemType> *next;
15 };
16
17 class FullQueue
18 {
19 };
20 class EmptyQueue
21 {
22 };
23
24 template <class ItemType>
25 class QueueType
26 {
27 public:
28     QueueType();
29     // Class constructor.
30     // Because there is a default constructor, the precondition
31     // that the queue has been initialized is omitted.
32     QueueType(int max);
33     // Parameterized class constructor.
34     ~QueueType();
35     // Class destructor.
36     QueueType(const QueueType &anotherQueue);
37     // Copy constructor
38     void MakeEmpty();
39     // Function: Initializes the queue to an empty state.
40     // Post: Queue is empty.
41     bool IsEmpty() const;
42     // Function: Determines whether the queue is empty.
43     // Post: Function value = (queue is empty)
44     bool IsFull() const;
45     // Function: Determines whether the queue is full.
46     // Post: Function value = (queue is full)
47     void Enqueue(ItemType newItem);
```

```
48 // Function: Adds newItem to the rear of the queue.
49 // Post: If (queue is full) FullQueue exception is thrown
50 //       else newItem is at rear of queue.
51 void Dequeue(ItemType &item);
52 // Function: Removes front item from the queue and returns it in item.
53 // Post: If (queue is empty) EmptyQueue exception is thrown
54 //       and item is undefined
55 //       else front element has been removed from queue and
56 //       item is a copy of removed element.
57 void Print();
58 // Function: Display the contents of the QueType in the console output.
59 // Post: The QueType remains unchanged.
60
61 private:
62     NodeType<ItemType> *front;
63     NodeType<ItemType> *rear;
64 };
65
66 template <class ItemType>
67 void QueType<ItemType>::Print()
68 {
69     NodeType<ItemType> *temp = front;
70     std::cout << "Front:";
71     while (temp != rear)
72     {
73         std::cout << temp->info << " ";
74         temp = temp->next;
75     }
76     std::cout << temp->info << ":Rear" << std::endl;
77 }
78
79 template <class ItemType>
80 QueType<ItemType>::QueType() // Class constructor.
81 // Post: front and rear are set to NULL.
82 {
83     front = nullptr;
84     rear = nullptr;
85 }
86
87 template <class ItemType>
88 void QueType<ItemType>::MakeEmpty()
89 // Post: Queue is empty; all elements have been deallocated.
90 {
91     NodeType<ItemType> *tempPtr;
92
93     while (front != nullptr)
94     {
95         tempPtr = front;
96         front = front->next;
```

```
97     delete tempPtr;
98 }
99 rear = nullptr;
100 }
101
102 // Class destructor.
103 template <class ItemType>
104 QueueType<ItemType>::~~QueueType()
105 {
106     MakeEmpty();
107 }
108
109 template <class ItemType>
110 bool QueueType<ItemType>::IsFull() const
111 // Returns true if there is no room for another ItemType
112 // on the free store; false otherwise.
113 {
114     NodeType<ItemType> *location;
115     try
116     {
117         location = new NodeType<ItemType>;
118         delete location;
119         return false;
120     }
121     catch (std::bad_alloc &)
122     {
123         return true;
124     }
125 }
126
127 template <class ItemType>
128 bool QueueType<ItemType>::IsEmpty() const
129 // Returns true if there are no elements on the queue; false otherwise.
130 {
131     return (front == nullptr);
132 }
133
134 template <class ItemType>
135 void QueueType<ItemType>::Enqueue(ItemType newItem)
136 // Adds newItem to the rear of the queue.
137 // Pre: Queue has been initialized.
138 // Post: If (queue is not full) newItem is at the rear of the queue;
139 //        otherwise a FullQueue exception is thrown.
140
141 {
142     if (IsFull())
143         throw FullQueue();
144     else
145     {
```

```
146     NodeType<ItemType> *newNode;
147
148     newNode = new NodeType<ItemType>;
149     newNode->info = newItem;
150     newNode->next = nullptr;
151     if (rear == nullptr)
152         front = newNode;
153     else
154         rear->next = newNode;
155     rear = newNode;
156 }
157 }
158
159 template <class ItemType>
160 void QueueType<ItemType>::Dequeue(ItemType &item)
161 // Removes front item from the queue and returns it in item.
162 // Pre: Queue has been initialized and is not empty.
163 // Post: If (queue is not empty) the front of the queue has been
164 //        removed and a copy returned in item;
165 //        otherwise a EmptyQueue exception has been thrown.
166 {
167     if (IsEmpty())
168         throw EmptyQueue();
169     else
170     {
171         NodeType<ItemType> *tempPtr;
172         tempPtr = front;
173         item = front->info;
174         front = front->next;
175         if (front == nullptr)
176             rear = nullptr;
177         delete tempPtr;
178     }
179 }
180
181 template <class ItemType>
182 QueueType<ItemType>::QueueType(const QueueType &anotherQueue)
183 {
184     NodeType<ItemType> *ptr1;
185     NodeType<ItemType> *ptr2;
186
187     if (anotherQueue.front == nullptr)
188         front = nullptr;
189     else
190     {
191         front = new NodeType<ItemType>;
192         front->info = anotherQueue.front->info;
193         ptr1 = anotherQueue.front->next;
194         ptr2 = front;
```

```
195     while (ptr1 != nullptr)
196     {
197         ptr2->next = new NodeType<ItemType>;
198         ptr2 = ptr2->next;
199         ptr2->info = ptr1->info;
200         ptr1 = ptr1->next;
201     }
202     ptr2->next = NULL;
203     rear = ptr2;
204 }
205 }
206 #endif
207
```